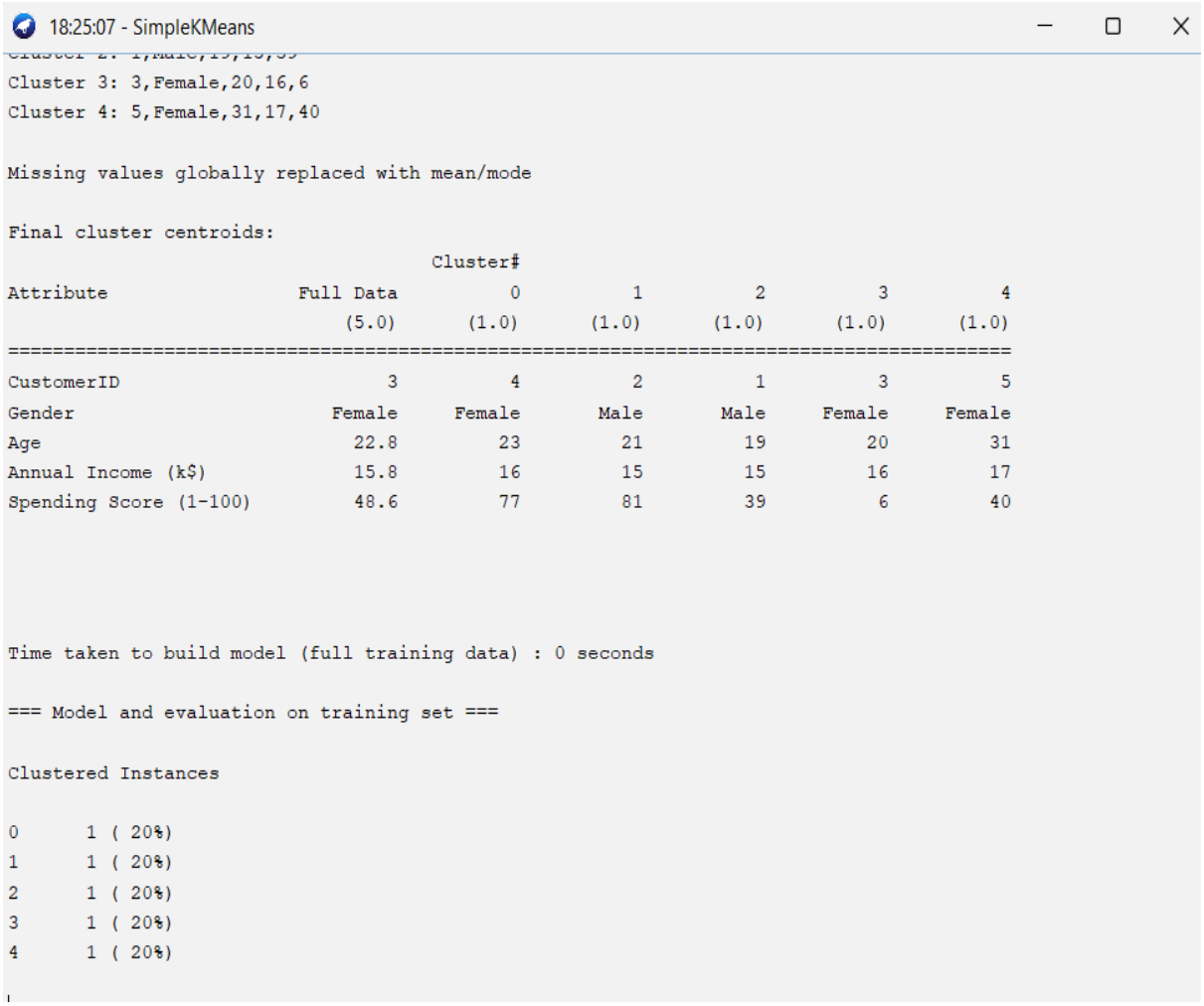
DATA WAREHOUSING AND DATA MINING

DAY - 4 LAB PROGRAMS

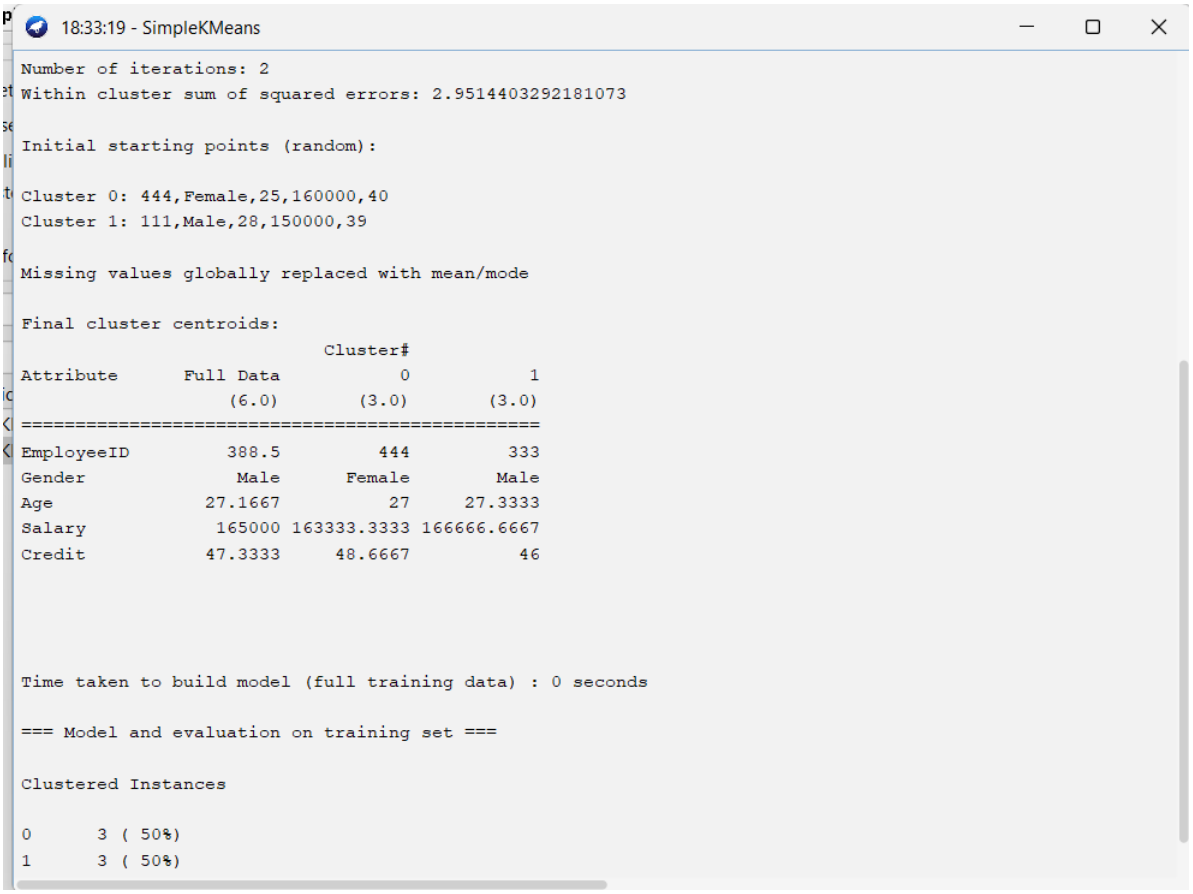
Question – 1:

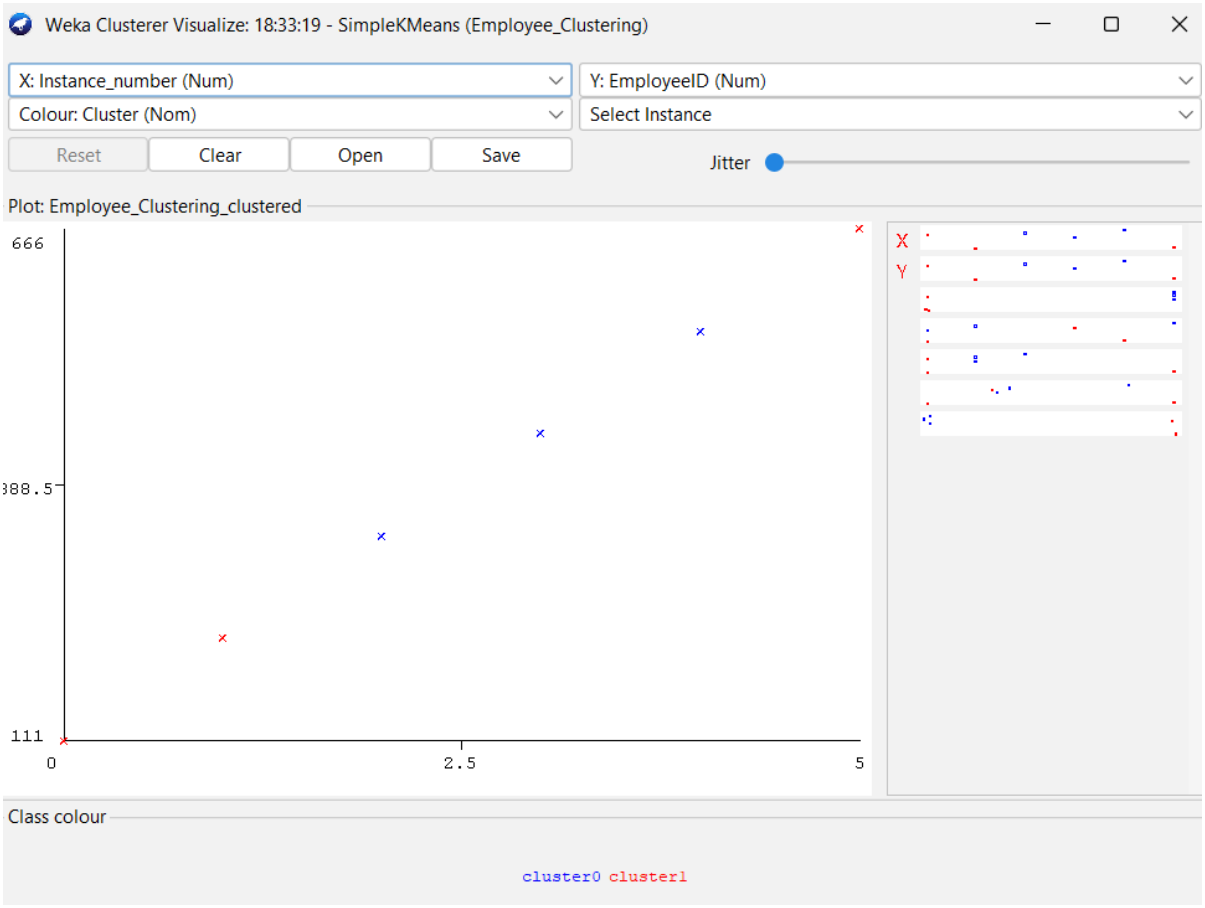
Output:



Question – 2:

Output:





Question – 3:

Output:

18:40:14 - bayes.NaiveBayes

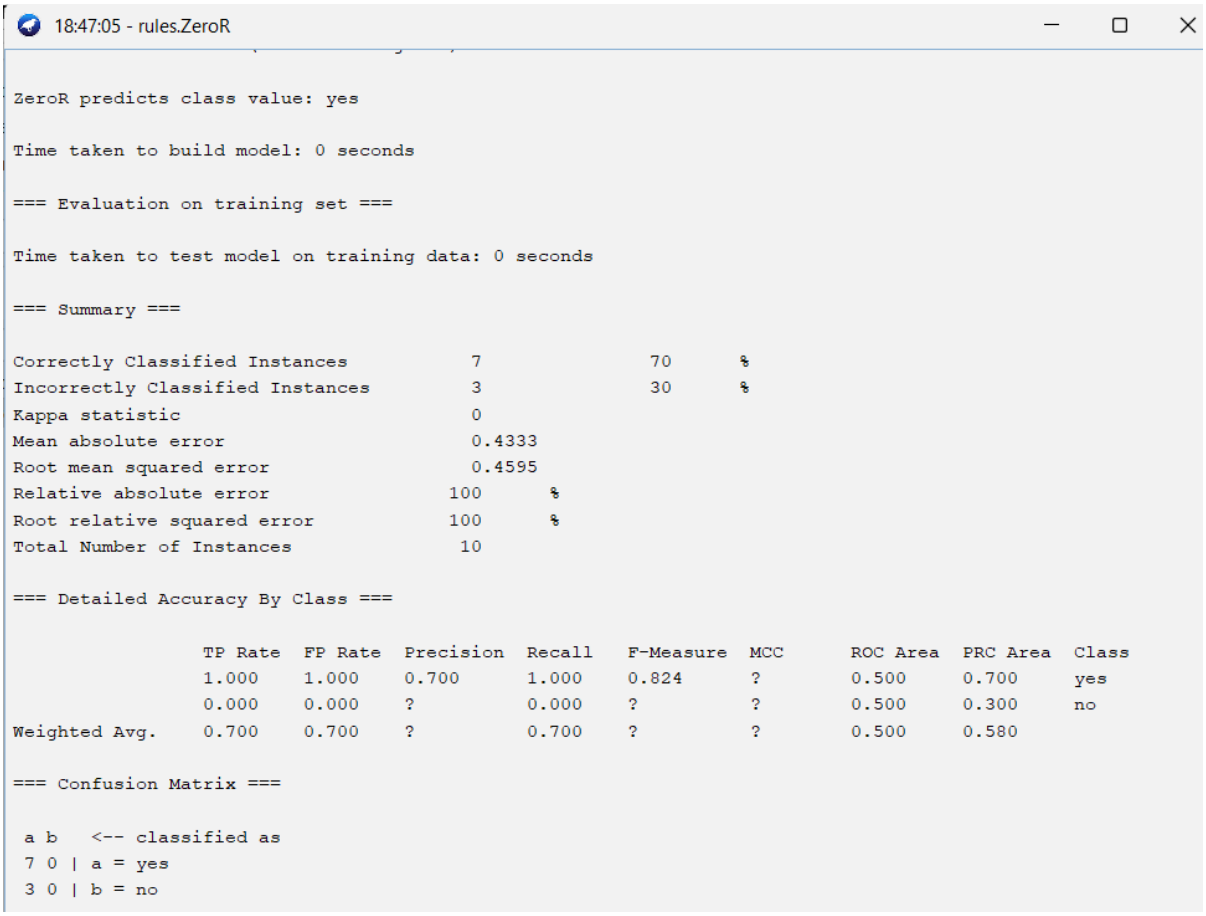
```
Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances          13               92.8571 %
Incorrectly Classified Instances         1                7.1429 %
Kappa statistic                          0.8372
Mean absolute error                      0.2917
Root mean squared error                  0.3392
Relative absolute error                 62.8233 %
Root relative squared error             70.7422 %
Total Number of Instances               14

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.800    0.000    1.000      0.800    0.889      0.849    0.911     0.911     No
                 1.000    0.200    0.900      1.000    0.947      0.849    0.922     0.947     Yes
Weighted Avg.    0.929    0.129    0.936      0.929    0.926      0.849    0.918     0.934

=== Confusion Matrix ===

 a b   <-- classified as
 4 1 | a = No
 0 9 | b = Yes
```

18:42:13 - functions.SMO

```
Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances          12               85.7143 %
Incorrectly Classified Instances         2               14.2857 %
Kappa statistic                          0.6585
Mean absolute error                      0.1429
Root mean squared error                  0.378
Relative absolute error                 30.7692 %
Root relative squared error             78.8263 %
Total Number of Instances               14

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.600    0.000    1.000      0.600    0.750      0.701    0.800     0.743     No
                 1.000    0.400    0.818      1.000    0.900      0.701    0.800     0.818     Yes
Weighted Avg.    0.857    0.257    0.883      0.857    0.846      0.701    0.800     0.791

=== Confusion Matrix ===

 a b   <-- classified as
 3 2 | a = No
 0 9 | b = Yes
```

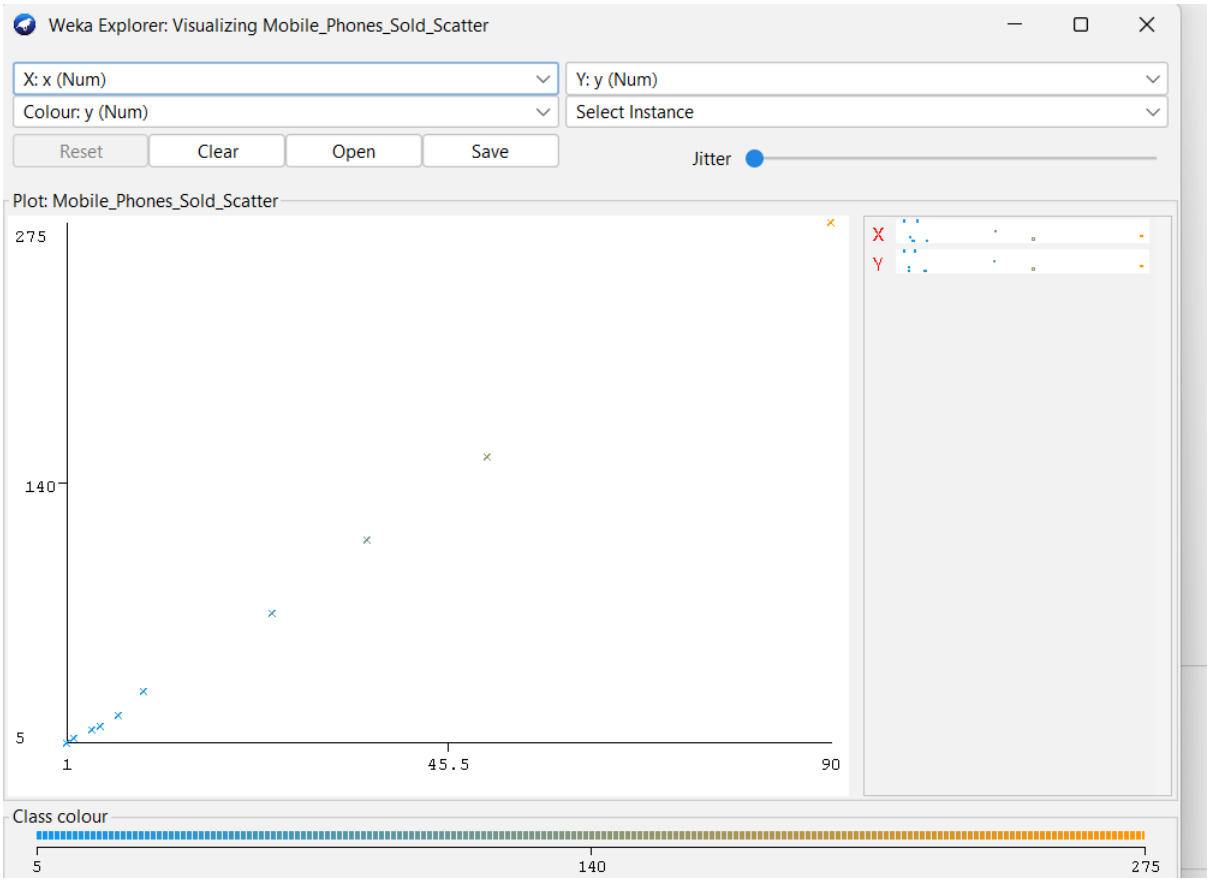
Question – 4:

Output:



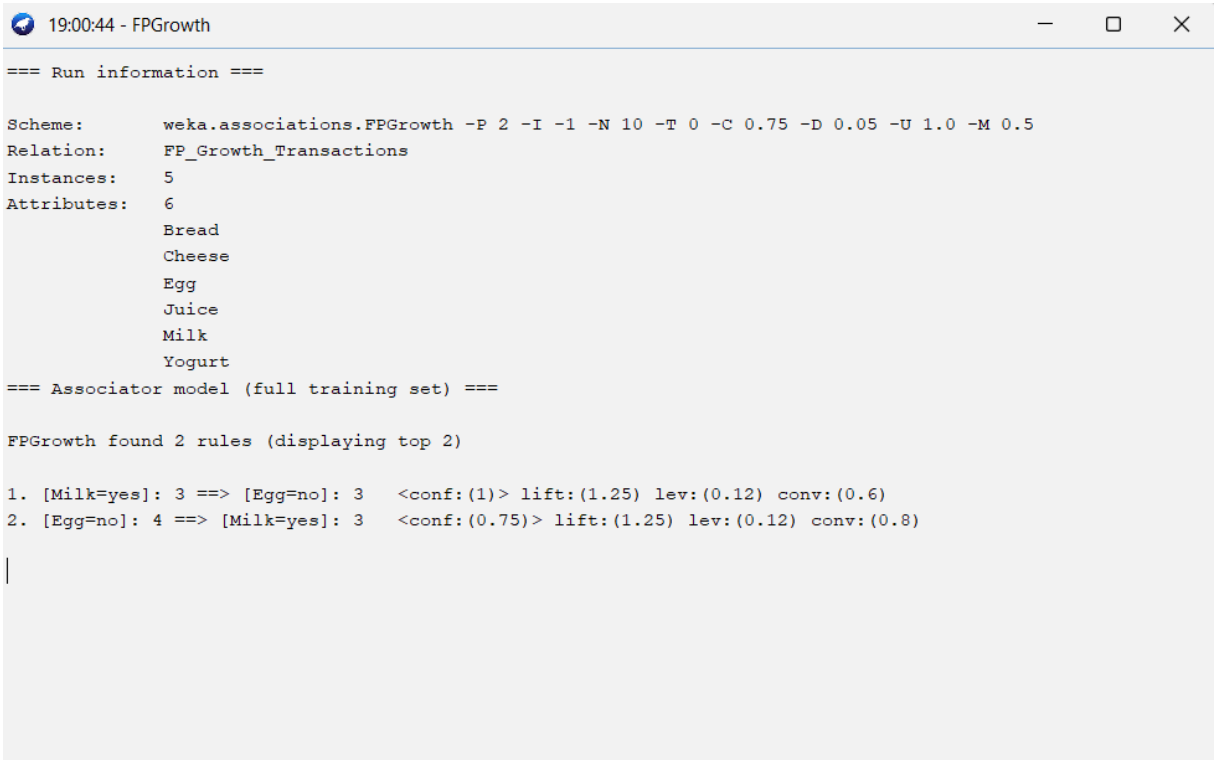
Question – 5:

Output:



Question – 6:

Output:



Question - 7

Perform Diabetes Data prediction using a Decision Tree classifier in WEKA and compare its performance with the Support Vector Machine (SVM) classifier. Calculate and compare the accuracy and F1-measure, plot the results, and explain the summary.

Aim

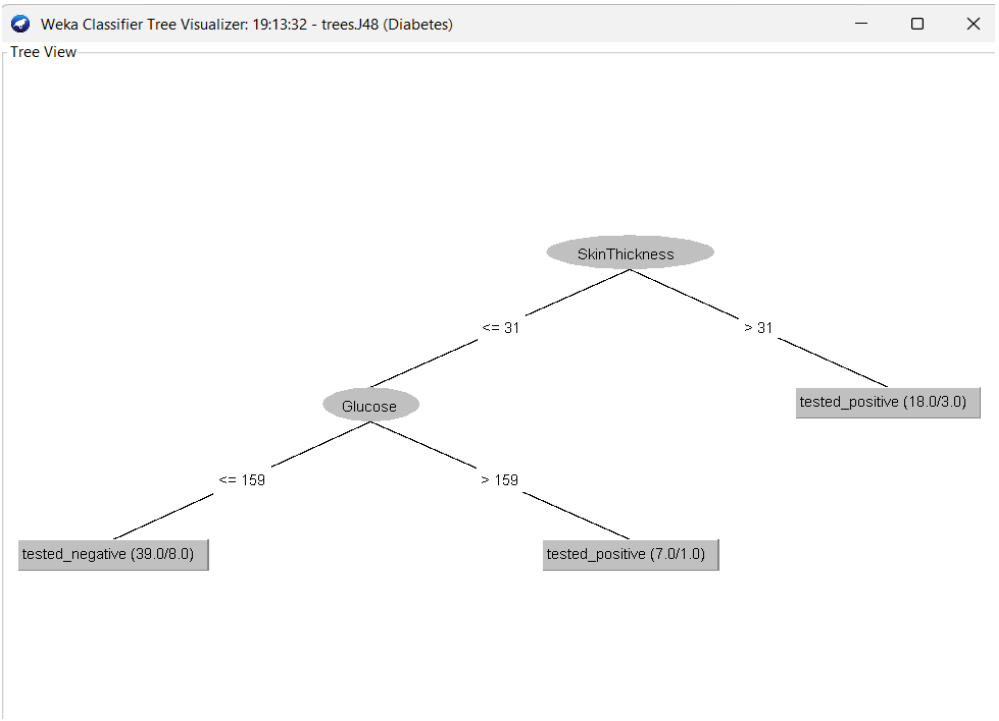
To predict diabetes using a Decision Tree classifier in WEKA and compare its performance with SVM based on accuracy and F1-measure.

Steps

1. Open Weka → Explorer → Preprocess and load the Diabetes dataset.
2. Go to Classify → Choose → trees → J48 and select the diabetes outcome as the class attribute.
3. Click Start and record the accuracy, precision, recall, and F1-measure.
4. Select functions → SMO (SVM) and click Start.
5. Record the accuracy and F1-measure for SVM.
6. Compare the J48 and SVM results using a table.
7. Plot a graph comparing Accuracy and F1-measure of both classifiers.

8. Explain which classifier gives better performance.

Output:



19:14:52 - functions.SMO

```
Time taken to build model: 0.09 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances          47               73.4375 %
Incorrectly Classified Instances        17               26.5625 %
Kappa statistic                          0.456
Mean absolute error                      0.2656
Root mean squared error                  0.5154
Relative absolute error                 53.5817 %
Root relative squared error            103.5332 %
Total Number of Instances               64

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.829    0.379    0.725      0.829    0.773      0.462    0.725     0.694     tested_negat
                 0.621    0.171    0.750      0.621    0.679      0.462    0.725     0.637     tested_posit
Weighted Avg.    0.734    0.285    0.736      0.734    0.731      0.462    0.725     0.669

=== Confusion Matrix ===

  a  b   <-- classified as
 29  6 |  a = tested_negative
 11 18 |  b = tested_positive
```

Question – 8:

Implement an R script to partition the given student marks into three bins using (a) Equal-Frequency (Equi-Depth), (b) Equal-Width, and (c) Clustering methods, and plot the data using histograms.

Aim

To partition student marks into three bins using different discretization methods and visualize the results using histograms.

R Code

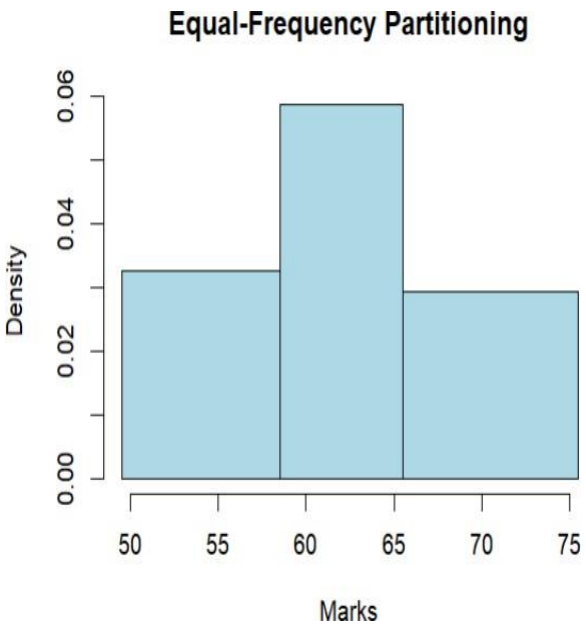
```
marks <- c(55,60,71,63,55,65,50,55,58,59,61,63,65,67,71,72,75)


marks <- sort(marks)


k <- 3

n <- length(marks)


eq_freq <- cut(

  seq_along(marks),

  breaks = c(0, ceiling(n/3), ceiling(2*n/3), n),

  labels = c("Bin 1", "Bin 2", "Bin 3")

)


print(data.frame(Marks = marks, Bin = eq_freq))


hist(marks,

     breaks = c(49.5,58.5,65.5,75.5),

     main = "Equal-Frequency Partitioning",

     xlab = "Marks")



min_mark <- min(marks)

max_mark <- max(marks)


breaks_width <- seq(min_mark, max_mark, length.out = k + 1)
```

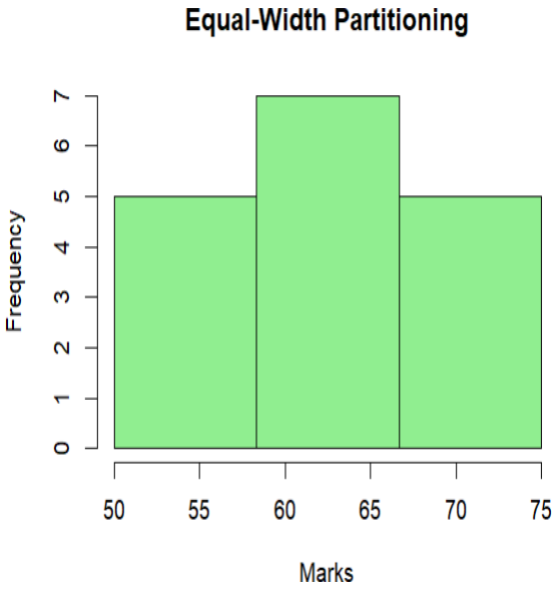
```
eq_width <- cut(
  marks,
  breaks = breaks_width,
  include.lowest = TRUE,
  labels = c("Bin 1", "Bin 2", "Bin 3")
)

print(data.frame(Marks = marks, Bin = eq_width))

hist(marks,
     breaks = breaks_width,
     main = "Equal-Width Partitioning",
     xlab = "Marks")



set.seed(123)

km <- kmeans(marks, centers = 3)

clustered_data <- data.frame(
  Marks = marks,
  Cluster = km$cluster
)

print(clustered_data)
print(km$centers)

hist(marks,
     breaks = 10,
     main = "Clustering Partitioning",
```

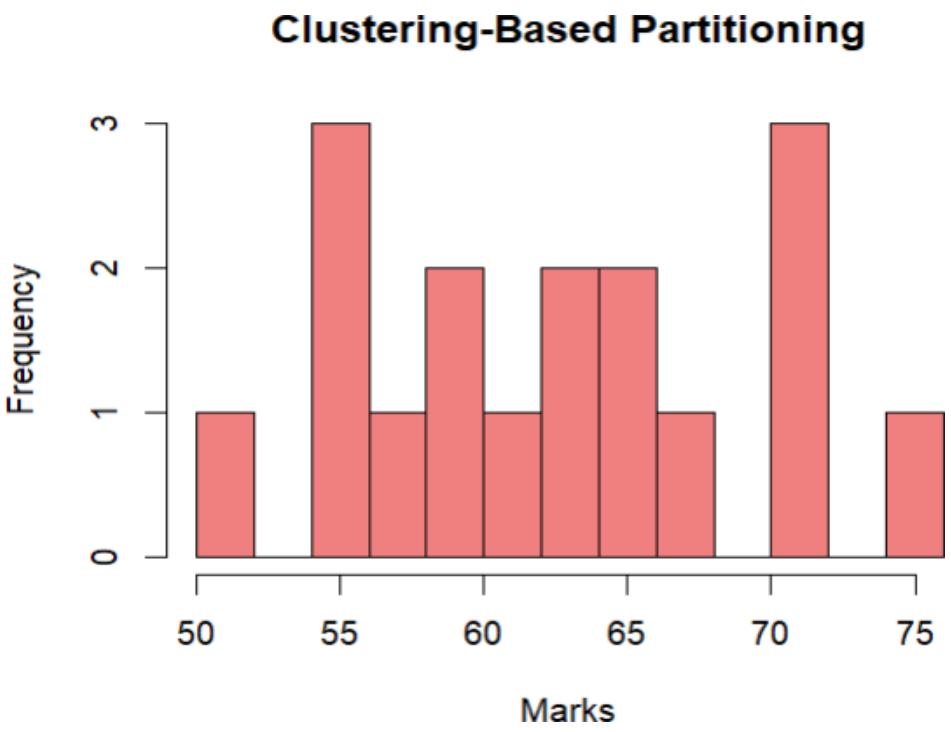
xlab = "Marks")

Output:

```
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
+       breaks = c(49.5,58.5,65.5,75.5),
+       main = "Equal-Frequency Partitioning",
+       xlab = "Marks")
>
>
> min_mark <- min(marks)
> max_mark <- max(marks)
>
> breaks_width <- seq(min_mark, max_mark, length.out = k + 1)
>
> eq_width <- cut(
+   marks,
+   breaks = breaks_width,
+   include.lowest = TRUE,
+   labels = c("Bin 1", "Bin 2", "Bin 3")
+ )
>
> print(data.frame(Marks = marks, Bin = eq_width))
   Marks   Bin
1     50 Bin 1
2     55 Bin 1
3     55 Bin 1
4     55 Bin 1
5     58 Bin 1
6     59 Bin 2
7     60 Bin 2
8     61 Bin 2
9     63 Bin 2
10    63 Bin 2
11    65 Bin 2
12    65 Bin 2
13    67 Bin 3
14    71 Bin 3
15    71 Bin 3
16    72 Bin 3
17    75 Bin 3
>
```







Question - 9

Consider the given Decision Tree based on Height, Weight, and Gender.
a) Create the dataset in ARFF format and calculate the accuracy and decision.
b) Generate rules using rule-based induction.
c) Compare both algorithms and plot the confusion matrix.

Aim

To implement the given Decision Tree in WEKA, calculate its accuracy, generate classification rules using rule-based induction, and compare the results using a confusion matrix.

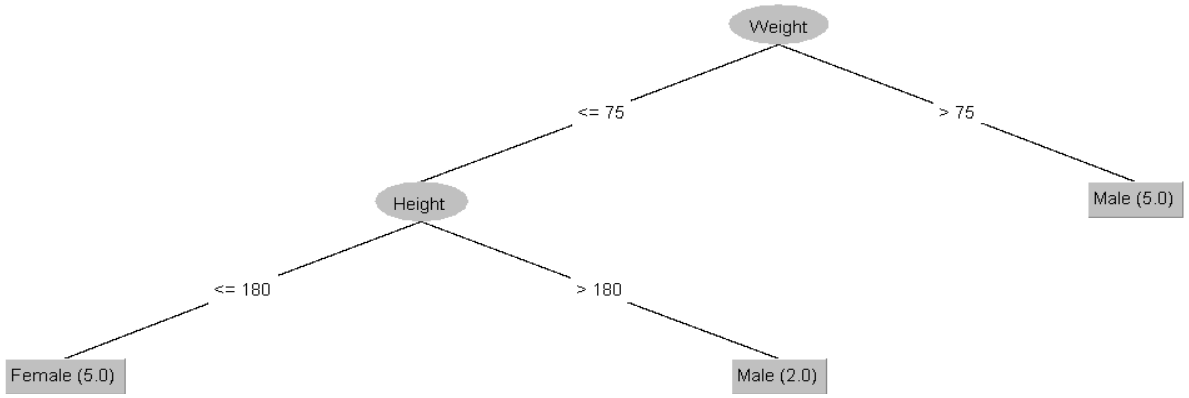
Steps:

1. Open WEKA → Explorer → Preprocess and load Height_Weight_Gender.arff.
2. Go to Classify → Choose → trees → J48 and select Gender as the class.
3. Click Start and record the decision tree and accuracy.
4. Go to Classify → Choose → rules → JRip and click Start to generate rules.
5. Compare the J48 and JRip results.
6. Use the confusion matrix from the output to compare the

algorithms. Output:

Weka Classifier Tree Visualizer: 19:28:59 - trees.J48 (Height_Weight_Gender)

Tree View



19:28:59 - trees.J48

```
Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0.01 seconds

=== Summary ===

Correctly Classified Instances          12               100      %
Incorrectly Classified Instances         0                 0      %
Kappa statistic                          1
Mean absolute error                      0
Root mean squared error                  0
Relative absolute error                  0      %
Root relative squared error              0      %
Total Number of Instances               12

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Male
                 1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Female
Weighted Avg.    1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000

=== Confusion Matrix ===

 a b   <-- classified as
 7 0 | a = Male
 0 5 | b = Female
```

19:29:47 - rules.JRip

```
Number of Rules : 3


Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances          12               100      %
Incorrectly Classified Instances         0                 0      %
Kappa statistic                          1
Mean absolute error                      0
Root mean squared error                  0
Relative absolute error                  0      %
Root relative squared error              0      %
Total Number of Instances               12

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Male
                 1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Female
Weighted Avg.    1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000

=== Confusion Matrix ===

 a b   <-- classified as
 7 0 | a = Male
 0 5 | b = Female
```

Question – 10:

Create an ARFF file for the given transaction dataset and implement both the Apriori and FP-Growth algorithms in WEKA. Compare the rules generated by both algorithms and identify the unique rules generated by each algorithm.

Given:

- Minimum Support = 2 transactions
- Minimum Confidence = 50%

Aim

To generate and compare association rules using Apriori and FP-Growth algorithms in WEKA and identify the unique rules generated by each algorithm.

Steps

1. Create the transaction dataset in ARFF format and load it in WEKA → Explorer → Preprocess.
2. Go to Associate → Choose → Apriori.
3. Set Minimum Support = 2/9 ≈ 22.22% and Minimum Confidence = 50%, then click Start.
4. Select FP-Growth and use the same support and confidence values, then click Start.
5. Record the rules generated by both algorithms.
6. Compare the two sets of rules and identify the common and unique rules.

Output:

19:38:16 - Apriori

```
Apriori
=======

Minimum support: 0.27 (2 instances)
Minimum metric <confidence>: 0.5
Number of cycles performed: 15

Generated sets of large itemsets:

Size of set of large itemsets L(1): 10

Size of set of large itemsets L(2): 27

Size of set of large itemsets L(3): 30

Size of set of large itemsets L(4): 13

Size of set of large itemsets L(5): 2

Best rules found:

 1. BPL=no 2 ==> SONY=yes 2    <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
 2. LG=yes 2 ==> SONY=yes 2    <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
 3. LG=yes 2 ==> BPL=yes 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
 4. SAMSUNG=yes 2 ==> BPL=yes 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
 5. BPL=no 2 ==> LG=no 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
 6. BPL=no 2 ==> SAMSUNG=no 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
 7. BPL=no 2 ==> ONIDA=yes 2    <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
 8. LG=yes 2 ==> SAMSUNG=no 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
 9. SAMSUNG=yes 2 ==> LG=no 2    <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
10. SAMSUNG=yes 2 ==> ONIDA=no 2    <conf:(1)> lift:(3) lev:(0.15) [1] conv:(1.33)
```

19:39:14 - FPGrowth

```
=== Run information ===

Scheme:       weka.associations.FPGrowth -P 2 -I -1 -N 10 -T 0 -C 0.5 -D 0.05 -U 0.22 -M 0.22
Relation:     Sony_BPL_Association
Instances:    9
Attributes:   5
              SONY
              BPL
              LG
              SAMSUNG
              ONIDA
=== Associator model (full training set) ===

FPGrowth found 10 rules (displaying top 10)

 1. [BPL=no]: 2 ==> [SAMSUNG=no]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 2. [BPL=no]: 2 ==> [LG=no]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 3. [SAMSUNG=no, SONY=no]: 2 ==> [LG=no]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 4. [BPL=no]: 2 ==> [SAMSUNG=no, LG=no]: 2   <conf:(1)> lift:(1.8) lev:(0.1) conv:(0.89)
 5. [SAMSUNG=no, BPL=no]: 2 ==> [LG=no]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 6. [LG=no, BPL=no]: 2 ==> [SAMSUNG=no]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 7. [SONY=no]: 3 ==> [SAMSUNG=no]: 2   <conf:(0.67)> lift:(0.86) lev:(-0.04) conv:(0.33)
 8. [ONIDA=no]: 3 ==> [LG=no]: 2   <conf:(0.67)> lift:(0.86) lev:(-0.04) conv:(0.33)
 9. [SONY=no]: 3 ==> [SAMSUNG=no, LG=no]: 2   <conf:(0.67)> lift:(1.2) lev:(0.04) conv:(0.67)
10. [LG=no, SONY=no]: 3 ==> [SAMSUNG=no]: 2   <conf:(0.67)> lift:(0.86) lev:(-0.04) conv:(0.33)
```

Question – 11:

Given the strike-rates scored by a batsman in different tournaments: 100, 70, 60, 90, 90, perform:

a) Min-Max normalization with range 0 to 1
b) Z-score normalization
c) Z-score normalization using mean absolute deviation
d) Decimal scaling normalization

Aim

To perform different data normalization techniques on the given batsman's strike-rate data using R.

R-Code:

```
x <- c(100, 70, 60, 90, 90)


min_max <- (x - min(x)) / (max(x) - min(x))


mean_x <- mean(x)

sd_x <- sd(x)

z_score <- (x - mean_x) / sd_x


mad_value <- mean(abs(x - mean_x))

z_mad <- (x - mean_x) / mad_value


j <- ceiling(log10(max(abs(x)) + 1))

decimal_scaling <- x / (10^j)


print("Original Data:")

print(x)


print("Min-Max Normalization:")

print(min_max)


print("Z-Score Normalization:")

print(z_score)
```

```
print("Z-Score using Mean Absolute Deviation:")
print(z_mad)

print("Decimal Scaling Normalization:")
print(decimal_scaling)
```

Output:

```
Console   Terminal   Background Jobs
R 4.5.2 · ~/
> x <- c(100, 70, 60, 90, 90)
>
> min_max <- (x - min(x)) / (max(x) - min(x))
>
> mean_x <- mean(x)
> sd_x <- sd(x)
> z_score <- (x - mean_x) / sd_x
>
> mad_value <- mean(abs(x - mean_x))
> z_mad <- (x - mean_x) / mad_value
>
> j <- ceiling(log10(max(abs(x)) + 1))
> decimal_scaling <- x / (10^j)
>
> print("Original Data:")
[1] "Original Data:"
> print(x)
[1] 100  70  60  90  90
>
> print("Min-Max Normalization:")
[1] "Min-Max Normalization:"
> print(min_max)
[1] 1.00 0.25 0.00 0.75 0.75
>
> print("Z-Score Normalization:")
[1] "Z-Score Normalization:"
> print(z_score)
[1]  1.0954451 -0.7302967 -1.3388774  0.4868645  0.4868645
>
> print("Z-Score using Mean Absolute Deviation:")
[1] "Z-Score using Mean Absolute Deviation:"
> print(z_mad)
[1]  1.3235294 -0.8823529 -1.6176471  0.5882353  0.5882353
```

Question - 12

Suppose some cars are tested for AvgSpeed and TotalTime for 9 randomly selected cars. Calculate:

a) The standard deviation of AvgSpeed and TotalTime.
b) The variance of AvgSpeed and TotalTime.

Aim

To calculate the standard deviation and variance of AvgSpeed and TotalTime using R.

R-Code:

```
AvgSpeed <- c(78,81,82,74,83,82,77,80,70)

TotalTime <- c(39,37,36,42,35,36,40,38,46)

sd_AvgSpeed <-
sd(AvgSpeed) sd_TotalTime
<- sd(TotalTime)

var_AvgSpeed <-
var(AvgSpeed) var_TotalTime
<- var(TotalTime)

print("Standard Deviation:")
print(sd_AvgSpeed)
print(sd_TotalTime)

print("Variance:")
print(var_AvgSpeed)
print(var_TotalTime)
```

Output:

```
> AvgSpeed <- c(78,81,82,74,83,82,77,80,70)
>
> TotalTime <- c(39,37,36,42,35,36,40,38,46)
>
> sd_AvgSpeed <- sd(AvgSpeed)
> sd_TotalTime <- sd(TotalTime)
>
> var_AvgSpeed <- var(AvgSpeed)
> var_TotalTime <- var(TotalTime)
>
> print("Standard Deviation:")
[1] "Standard Deviation:"
> print(sd_AvgSpeed)
[1] 4.304391
> print(sd_TotalTime)
[1] 3.492054
>
> print("Variance:")
[1] "Variance:"
> print(var_AvgSpeed)
[1] 18.52778
> print(var_TotalTime)
[1] 12.19444
>
```

Question – 13:

Consider the given transaction table containing items bought by five customers.
a) Find all frequent itemsets using Apriori and FP-Growth and compare the efficiency of the two mining processes.
b) List all strong association rules satisfying the given rule:

buys(X, item1) ∧ buys(X, item2) ⇒ buys(X, item3)

along with their support and confidence.

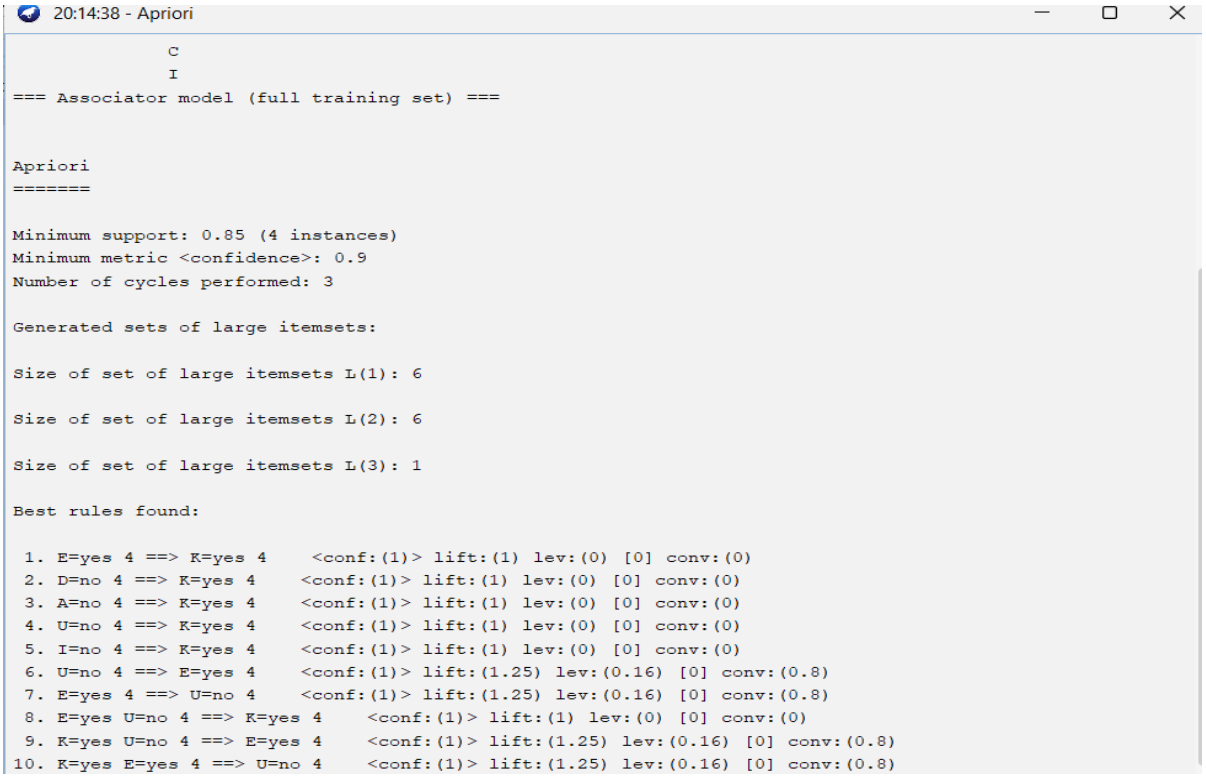
Aim

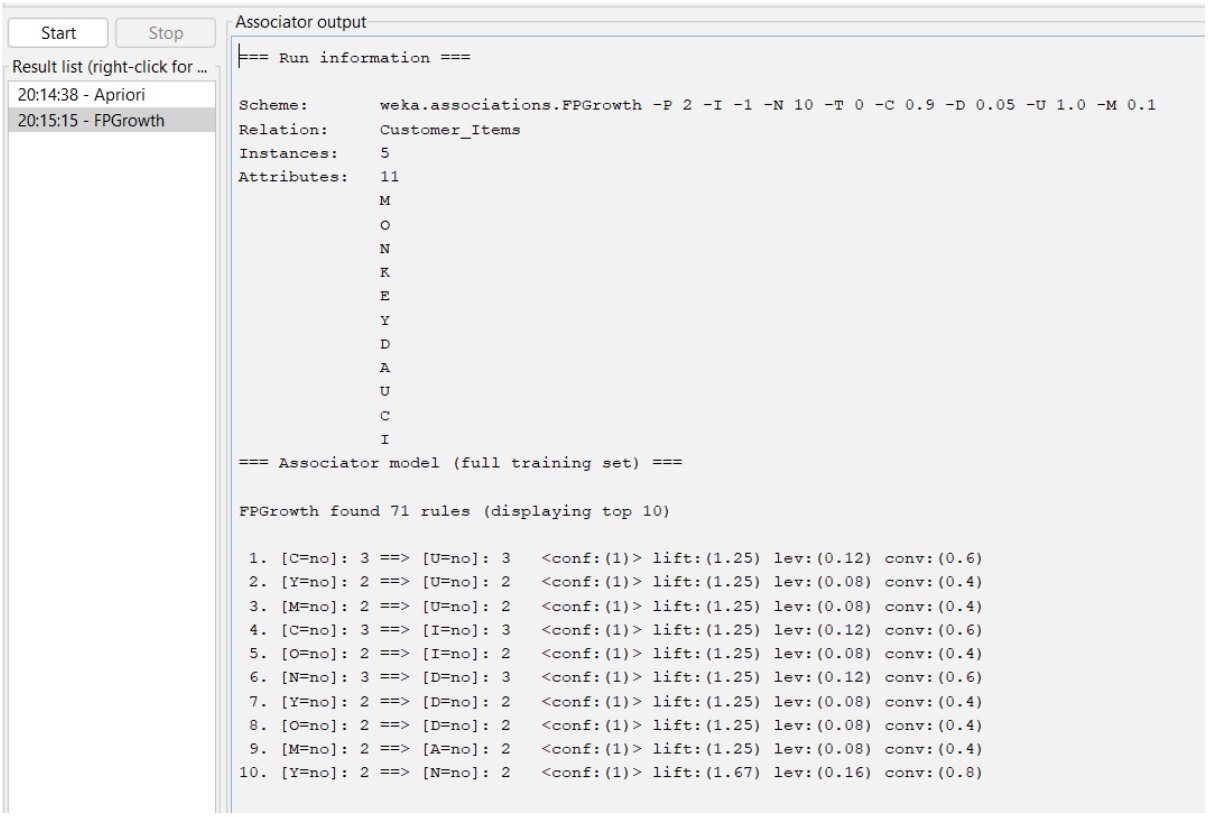
To find frequent itemsets and generate strong association rules from transaction data using Apriori and FP-Growth algorithms in WEKA, and compare their results.

Steps:

1. Open WEKA → Explorer → Preprocess and load Customer_Items_T13.arff.
2. Go to Associate → Choose → Apriori.
3. Set the required minimum support and confidence values given by your teacher, then click Start.
4. Record the frequent itemsets and association rules.
5. Select FP-Growth and use the same support and confidence settings.
6. Click Start and record its frequent itemsets and rules.
7. Compare the rules produced by Apriori and FP-Growth and identify the common and unique rules.
8. Calculate the support and confidence of the strong rules matching item1 ∧ item2 → item3.

Output:




Start
Stop
Result list (right-click for ...
20:14:38 - Apriori
20:15:15 - FPGrowth
Associator output
=== Run information ===
Scheme: weka.associations.FPGrowth -P 2 -I -1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1
Relation: Customer_Items
Instances: 5
Attributes: 11
M
O
N
K
E
Y
D
A
U
C
I
=== Associator model (full training set) ===
FPGrowth found 71 rules (displaying top 10)
1. [C=no]: 3 ==> [U=no]: 3 <conf:(1)> lift:(1.25) lev:(0.12) conv:(0.6)
2. [Y=no]: 2 ==> [U=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
3. [M=no]: 2 ==> [U=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
4. [C=no]: 3 ==> [I=no]: 3 <conf:(1)> lift:(1.25) lev:(0.12) conv:(0.6)
5. [O=no]: 2 ==> [I=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
6. [N=no]: 3 ==> [D=no]: 3 <conf:(1)> lift:(1.25) lev:(0.12) conv:(0.6)
7. [Y=no]: 2 ==> [D=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
8. [O=no]: 2 ==> [D=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
9. [M=no]: 2 ==> [A=no]: 2 <conf:(1)> lift:(1.25) lev:(0.08) conv:(0.4)
10. [Y=no]: 2 ==> [N=no]: 2 <conf:(1)> lift:(1.67) lev:(0.16) conv:(0.8)